

Installation

```
%%capture
import os, importlib.util
!pip install --upgrade -qqq uv
if importlib.util.find_spec("torch") is None or "COLAB_" in "".join(os.environ.keys()):
    try: import numpy, PIL; _numpy = f"numpy=={numpy.__version__}"; _pil = f"pillow=={PIL.__version__}"
    except: _numpy = "numpy"; _pil = "pillow"
    !uv pip install -qqq \
        "torch==2.8.0" "triton==3.3.0" {_numpy} {_pil} torchvision bitsandbytes xformers==0.0.32.post2 \
        "unsloth_zoo[base] @ git+https://github.com/unslothai/unsloth-zoo" \
        "unsloth[base] @ git+https://github.com/unslothai/unsloth"
    !uv pip install -qqq --no-deps "torchcodecs==0.7.0"
elif importlib.util.find_spec("unsloth") is None:
    !uv pip install -qqq unsloth
!uv pip install --upgrade --no-deps "tokenizers>=0.22.0,<=0.23.0" trl==0.22.2 unsloth unsloth_zoo
!uv pip install transformers==5.2.0
# Unsloth bundles the gated delta net kernels; a leftover pip fla would shadow them
!uv pip uninstall -qqq flash-linear-attention fla-core
# Prebuilt causal_conv1d when one exists for this torch, otherwise transformers' torch conv1d (no 10 minute build)
import sys, torch; _t = ".".join(torch.__version__.split(".")[:2]); _cu = (torch.version.cuda or "0").split(".")[0]; _py = f"cp{sys.version_info[0]}{sys.version_info[1]}{sys.version_info[2]}"
!uv pip install -qqq "https://github.com/Dao-AI-Lab/causal-conv1d/releases/download/v1.7.0/causal_conv1d-1.7.0+cu{_cu}torch{_t}.whl"
!uv pip install --no-deps --upgrade "torchao==0.16.0"
```

Unsloth

```
from unsloth import FastVisionModel # FastLanguageModel for LLMs
import torch

# 4bit pre quantized models we support for 4x faster downloading + no OOMs.
fourbit_models = [
    "unsloth/Llama-3.2-11B-Vision-Instruct-bnb-4bit", # Llama 3.2 vision support
    "unsloth/Llama-3.2-11B-Vision-bnb-4bit",
    "unsloth/Llama-3.2-90B-Vision-Instruct-bnb-4bit", # Can fit in a 80GB card!
    "unsloth/Llama-3.2-90B-Vision-bnb-4bit",

    "unsloth/Pixtral-12B-2409-bnb-4bit", # Pixtral fits in 16GB!
    "unsloth/Pixtral-12B-Base-2409-bnb-4bit", # Pixtral base model

    "unsloth/Qwen2-VL-2B-Instruct-bnb-4bit", # Qwen2 VL support
    "unsloth/Qwen2-VL-7B-Instruct-bnb-4bit",
    "unsloth/Qwen2-VL-72B-Instruct-bnb-4bit",

    "unsloth/llava-v1.6-mistral-7b-hf-bnb-4bit", # Any Llava variant works!
    "unsloth/llava-1.5-7b-hf-bnb-4bit",
] # More models at https://huggingface.co/unsloth

model, tokenizer = FastVisionModel.from_pretrained(
    "unsloth/Qwen3.5-4B",
    load_in_4bit = True, # Use 4bit to reduce memory use. False for 16bit LoRA.
    use_gradient_checkpointing = "unsloth", # True or "unsloth" for long context
)
```

/usr/local/lib/python3.13/dist-packages/unsloth/_gpu_init.py:110: UserWarning: Unsloth: torchaudio cannot initialise against this device. Disabling torchaudio if cuda mismatched()

🐍 Unsloth: Will patch your computer to enable 2x faster free finetuning.

🐍 Unsloth Zoo will now patch everything to make training faster!

==((====))== Unsloth 2026.9.14: Fast Qwen3_5 patching. Transformers: 5.2.0.

\\ \ /| Tesla T4. Num GPUs = 1. Max memory: 14.563 GB. Platform: Linux.

0'0/ \ / \ Torch: 2.8.0+cu128. CUDA: 7.5. CUDA Toolkit: 12.8. Triton: 3.4.0

\ \ / Bfloat16 = FALSE. FA [Xformers = 0.0.32.post2. FA2 = False]

"-_-_" / Free license: <http://github.com/unslothai/unsloth>

Unsloth: Fast downloading is enabled - ignore downloading bars which are red colored!

Unsloth: Using float16 precision for qwen3_5 won't work! Using float32.

Loading weights: 100%

723/723 [00:37<00:00, 556.28it/s, Materializing param=model.visual.pos_embed.weight]

We now add LoRA adapters for parameter efficient finetuning - this allows us to only efficiently train 1% of all parameters.

[NEW] We also support finetuning ONLY the vision part of the model, or ONLY the language part. Or you can select both! You can also select to finetune the attention or the MLP layers!

```
model = FastVisionModel.get_peft_model(
    model,
    finetune_vision_layers = True, # False if not finetuning vision layers
    finetune_language_layers = True, # False if not finetuning language layers
    finetune_attention_modules = True, # False if not finetuning attention layers
    finetune_mlp_modules = True, # False if not finetuning MLP layers

    r = 16, # The larger, the higher the accuracy, but might overfit
    lora_alpha = 16, # Recommended alpha == r at least
    lora_dropout = 0,
    bias = "none",
    random_state = 3407,
    use_rslora = False, # We support rank stabilized LoRA
    loftq_config = None, # And LoftQ
    # target_modules = "all-linear", # Optional now! Can specify a list if needed
)
```

WARNING:unsloth_zoo.log:Unsloth: Failed to register input-embedding hook for `model.base\_model.model.model.visual`: `get_input_e

▼ Data Prep

Chuẩn bị dữ liệu bài toán hình học (Geometry3K) kết hợp lời giải suy luận từng bước (Chain-of-Thought chưng cất từ Gemini).

You can access the dataset [here](#).

```
#Upload file
from google.colab import files
dataPath= files.upload()
```

Chọn tệp geometry3k_cot_sft.jsonl
geometry3k_cot_sft.jsonl(n/a) - 3421375 bytes, last modified: 5/10/2026 - 100% done
 Saving geometry3k cot sft.jsonl to geometry3k cot sft (1).jsonl

```
import json
import random
from datasets import load_dataset

# 1. Tải ảnh gốc từ geometry3k
print("Đang nạp ảnh từ geometry3k...")
raw_hf_dataset = load_dataset("hiyouga/geometry3k", split="train")

# 2. Đọc file jsonl đã upload
jsonl_file = "geometry3k_cot_sft.jsonl"
with open(jsonl_file, "r", encoding="utf-8") as f:
    distilled_data = [json.loads(line) for line in f if line.strip()]

print(f"Tổng số mẫu chưng cất: {len(distilled_data)}")

# 3. Ghép ảnh gốc với dữ liệu CoT
formatted_samples = []
for item in distilled_data:
    idx = item["id"]

    raw_img = raw_hf_dataset[idx].get("images") or raw_hf_dataset[idx].get("image")
    img = raw_img[0] if isinstance(raw_img, list) else raw_img

    if img.mode != "RGB":
        img = img.convert("RGB")

    formatted_samples.append({
        "messages": [
            {
                "role": "user",
                "content": [
                    {"type": "text", "text": item["problem"]},
                    {"type": "image", "image": img}
                ]
            }
        ]
    })
```

```

    },
    {
        "role": "assistant",
        "content": [
            {"type": "text", "text": item["cot_solution"]}
        ]
    }
]
})

# 4. Trộn ngẫu nhiên và chia 90% Train - 10% Test trực tiếp trên List
random.seed(3407)
random.shuffle(formatted_samples)

split_idx = int(len(formatted_samples) * 0.9)
train_dataset = formatted_samples[:split_idx]
eval_dataset = formatted_samples[split_idx:]

print(f"Đã chia xong: {len(train_dataset)} mẫu Train | {len(eval_dataset)} mẫu Test/Eval")

Đang nạp ảnh từ geometry3k...
Tổng số mẫu chung cất: 2101
Đã chia xong: 1890 mẫu Train | 211 mẫu Test/Eval

```

Let's take an overview look at the dataset. We shall see what the 3rd image is, and what caption it had.

```

# Xem cấu trúc mẫu đầu tiên
print(train_dataset[0]["messages"][0]["content"][0]["text"]) # Câu hỏi
print(train_dataset[0]["messages"][1]["content"][0]["text"]) # Lời giải CoT mẫu
train_dataset[0]["messages"][0]["content"][1]["image"]         # Hiển thị ảnh

```

[Hiện kết quả đã ẩn](#)

Let's first see before we do any finetuning what the model outputs for the first example!

```

FastVisionModel.for_inference(model)

# Lấy thử 1 bài toán từ tập test độc lập
test_sample = eval_dataset[1]["messages"]
test_image = test_sample[0]["content"][1]["image"]
test_prompt = test_sample[0]["content"][0]["text"]

messages = [
    {"role": "user", "content": [
        {"type": "image"},
        {"type": "text", "text": test_prompt}
    ]}
]
input_text = tokenizer.apply_chat_template(messages, add_generation_prompt=True)
inputs = tokenizer(
    test_image,
    input_text,
    add_special_tokens=False,
    return_tensors="pt"
).to("cuda")

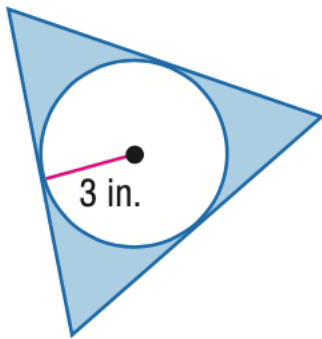
from transformers import TextStreamer
text_streamer = TextStreamer(tokenizer, skip_prompt=True)

print("--- KẾT QUẢ MÔ HÌNH GỐC (CHƯA FINE-TUNE) ---")
_ = model.generate(**inputs, streamer=text_streamer, max_new_tokens=3072,
                  use_cache=True, temperature=0.2)

```

[Hiện kết quả đã ẩn](#)

test_image



test_prompt

'<image>Find the area of the shaded region. Assume that all polygons that appear to be regular are regular. Round to the nearest tenth.'

✓ Train the model

Now let's train our model. We do 60 steps to speed things up, but you can set `num_train_epochs=1` for a full run, and turn off `max_steps=None`. We also support `DPOTrainer` and `GRPOTrainer` for reinforcement learning!

We use our new `UnslothVisionDataCollator` which will help in our vision finetuning setup.

```
from unsloth.trainer import UnslothVisionDataCollator
from trl import SFTTrainer, SFTConfig

FastVisionModel.for_training(model) # Chuyển mô hình sang chế độ huấn luyện!

trainer = SFTTrainer(
    model = model,
    tokenizer = tokenizer,
    data_collator = UnslothVisionDataCollator(model, tokenizer),
    train_dataset = train_dataset,
    eval_dataset = eval_dataset,          # Đánh giá trên tập test độc lập
    args = SFTConfig(
        # --- CẤU HÌNH BỘ NHỚ AN TOÀN CHO GPU T4 ---
        per_device_train_batch_size = 1, # 1 ảnh mỗi lượt nạp để tránh tràn VRAM
        gradient_accumulation_steps = 8, # Tích lũy 8 lượt -> Tổng batch vẫn = 8
        max_length = 1536,              # Đủ cho lời giải CoT mà không tốn RAM
        # -----

        warmup_steps = 10,
        num_train_epochs = 1,            # Chạy thử nghiệm 1 epoch (khoảng 237 steps)
        learning_rate = 2e-4,
        logging_steps = 5,               # Cứ 5 steps in log 1 lần
        optim = "adamw_8bit",
        weight_decay = 0.001,
        lr_scheduler_type = "cosine",
        seed = 3407,
        report_to = "none",

        # --- LƯU ĐỊNH KỲ ĐỂ RESUME KHI BỊ LỖI ---
        save_strategy = "steps",
        save_steps = 40,                 # Cứ 40 steps lưu 1 checkpoint
        save_total_limit = 2,            # Chỉ giữ 2 checkpoint gần nhất để đỡ đầy đĩa
        output_dir = "outputs_sft",
        # -----

        # --- ĐO VALIDATION LOSS CHO BÁO CÁO ---
        eval_strategy = "steps",
        eval_steps = 40,                 # Đo trên tập eval_dataset sau mỗi 40 steps
        # -----

        # Các tham số bắt buộc cho Vision Finetuning của Unsloth:
        remove_unused_columns = False,
        dataset_text_field = "",
        dataset_kwargs = {"skip_prepare_dataset": True},
    ),
)
```

Unsloth: Model does not have a default image size - using 512
Unsloth: Switching to float32 training since model cannot work with float16

> Show current memory stats

Hiện mã

GPU = Tesla T4. Max memory = 14.563 GB.
7.4 GB of memory reserved.

```
trainer_stats = trainer.train()

==(====)== Unsloth - 2x faster free finetuning | Num GPUs used = 1
  \ \ / |   Num examples = 1,890 | Num Epochs = 1 | Total steps = 237
0^0/ \_ / \   Batch size per device = 1 | Gradient accumulation steps = 8
 \      /     Data Parallel GPUs = 1 | Total batch size (1 x 8 x 1) = 8
"-_____"     Trainable parameters = 38,756,352 of 4,578,021,888 (0.85% trained)
[120/237 48:16 < 47:52, 0.04 it/s, Epoch 0.50/1]
```

Step	Training Loss	Validation Loss
40	0.348189	0.375665
80	0.363854	0.366686

Unsloth: Restored added_tokens_decoder metadata in outputs_sft/checkpoint-40/tokenizer_config.json.
Unsloth: Restored added_tokens_decoder metadata in outputs_sft/checkpoint-80/tokenizer_config.json.

> Show final memory and time stats

Hiện mã

559.2975 seconds used for training.
9.32 minutes used for training.
Peak reserved memory = 5.146 GB.
Peak reserved memory for training = 0.632 GB.

Peak reserved memory % of max memory = 35.336 %.
 Peak reserved memory for training % of max memory = 4.34 %.

▼ Inference

Test liền model vừa train (lưu ý bước này chỉ chạy sau khi thực hiện train model)

Nếu ở session mới thì buộc phải chạy code của phần Save Lora để nạp Lora vào

We use `min_p = 0.1` and `temperature = 1.5`. Read this [Tweet](#) for more information on why.

```
import io
from PIL import Image
from transformers import TextStreamer

# Kích hoạt chế độ suy luận nhanh
FastVisionModel.for_inference(model)

# Lấy 1 bài toán từ tập test độc lập
sample_idx = 0
sample = eval_dataset[sample_idx]["messages"]

user_content = sample[0]["content"]
problem_text = next(item["text"] for item in user_content if item["type"] == "text")
test_image = next(item["image"] for item in user_content if item["type"] == "image")
ground_truth = sample[1]["content"][0]["text"]

if isinstance(test_image, dict):
    if "bytes" in test_image and test_image["bytes"] is not None:
        test_image = Image.open(io.BytesIO(test_image["bytes"]))
    elif "path" in test_image and test_image["path"] is not None:
        test_image = Image.open(test_image["path"])

messages = [
    {
        "role": "user",
        "content": [
            {"type": "image"},
            {"type": "text", "text": problem_text}
        ]
    }
]

input_text = tokenizer.apply_chat_template(messages, add_generation_prompt=True)
inputs = tokenizer(
    test_image,
    input_text,
    add_special_tokens=False,
    return_tensors="pt"
).to("cuda")

print("=" * 60)
print(f"BÀI TOÁN THỬ NGHIỆM (Index: {sample_idx}): \n{problem_text}")
print("=" * 60)
print("MÔ HÌNH BẮT ĐẦU GIẢI TOÁN:")

text_streamer = TextStreamer(tokenizer, skip_prompt=True)
_ = model.generate(
    **inputs,
    streamer=text_streamer,
    max_new_tokens=1536,
    use_cache=True,
    temperature=0.2,
    top_p=0.95,
)

print("\n" + "=" * 60)
print(f"ĐÁP ÁN GỐC: \n{ground_truth}")
print("=" * 60)
```

$$H^{\wedge \{ \prime \}} = \backslash \beta N \int d \backslash \lambda \backslash \left\{ \backslash \frac { 1 } { 2 } \beta ^ { 2 } N ^ { 2 } \right\} \backslash \partial i a l _ { \{ \backslash \lambda \} } \backslash \zeta ^ { \backslash d a g g$$

✓ Saving, loading finetuned models

To save the final model as LoRA adapters, either use Hugging Face's `push_to_hub` for an online save or `save_pretrained` for a local save.

[NOTE] This ONLY saves the LoRA adapters, and not the full model. To save to 16bit or GGUF, scroll down!

Lệnh này chỉ lưu các trọng số LoRA mới học (dung lượng rất nhẹ, chỉ khoảng 50MB – 150MB) chứ không lưu toàn bộ 4 tỷ tham số của mô hình gốc. File này chính là checkpoint "ấm" (Warm-start) để bạn mang sang nạp vào Notebook GRPO (RL) ở giai đoạn sau.

```
model.save_pretrained("qwen3_5_4B_geometry_sft_lora") # Đổi tên cho đúng dự án
tokenizer.save_pretrained("qwen3_5_4B_geometry_sft_lora")

['qwen_lora/processor_config.json']
```

✓ Lưu drive file checkpoint

```
# Tùy chọn: Lưu thẳng vào Drive để không bao giờ mất
from google.colab import drive
drive.mount('/content/drive')
model.save_pretrained("/content/drive/MyDrive/qwen3_5_4B_geometry_sft_lora")
tokenizer.save_pretrained("/content/drive/MyDrive/qwen3_5_4B_geometry_sft_lora")
print("Đã lưu an toàn vào Google Drive!")
```

Now if you want to load the LoRA adapters we just saved for inference, set `False` to `True`:

Giải thích thêm:

Khi bạn chạy lệnh

```
model.save_pretrained("qwen_geometry_sft_lora")
```

Unsloth chỉ lưu phần trọng số thích ứng LoRA (dung lượng chỉ khoảng 50MB – 150MB), bao gồm các file cốt lõi:

- `adapter_model.safetensors`: Chứa các ma trận LoRA rank $r = 16$ vừa được huấn luyện.
- `adapter_config.json`: File cấu hình chỉ đường. Bên trong file này có một dòng quan trọng:


```
"base_model_name_or_path": "unsloth/Qwen3.5-4B"
```
- Các file cấu hình của tokenizer.

=> Thư mục này hoàn toàn không chứa 4 tỷ tham số của mô hình Qwen gốc.

Quá trình `from_pretrained("qwen_geometry_sft_lora")` diễn ra như thế nào?

Khi bạn đặt `RELOAD_FROM_DISK = True` và gọi lệnh trên, thư viện sẽ tự động thực hiện 2 bước ngấm:

1. Đọc file cấu hình: Unsloth đọc file `adapter_config.json` và thấy bạn dùng mô hình nền là `unsloth/Qwen3.5-4B`. Nó sẽ nạp mô hình nền gốc đó lên GPU (ở dạng lượng tử hóa 4-bit).
2. Gắn LoRA lên trên: Sau khi mô hình gốc đã nạp xong vào VRAM, Unsloth bốc các ma trận LoRA từ `adapter_model.safetensors` gắn đè lên các tầng Attention và MLP tương ứng.

```
import io
from PIL import Image
from transformers import TextStreamer

# 1. Đổi thành True nếu bạn mở session mới và muốn nạp lại LoRA từ đĩa
RELOAD_FROM_DISK = False

if RELOAD_FROM_DISK:
    from unsloth import FastVisionModel
    model, tokenizer = FastVisionModel.from_pretrained(
        model_name = "qwen_geometry_sft_lora", # Trỏ đúng thư mục đã lưu ở Cell 1
        load_in_4bit = True,
    )

# Kích hoạt chế độ suy luận nhanh
FastVisionModel.for_inference(model)

# 2. Lấy 1 bài toán hình học từ tập test đọc lập để kiểm chứng
sample = eval_dataset[0]["messages"]
user_content = sample[0]["content"]
```

```

test_prompt = next(item["text"] for item in user_content if item["type"] == "text")
test_image = next(item["image"] for item in user_content if item["type"] == "image")

# Giải nén an toàn nếu ảnh ở dạng dict/bytes
if isinstance(test_image, dict):
    if "bytes" in test_image and test_image["bytes"] is not None:
        test_image = Image.open(io.BytesIO(test_image["bytes"]))
    elif "path" in test_image and test_image["path"] is not None:
        test_image = Image.open(test_image["path"])

messages = [
    {
        "role": "user",
        "content": [
            {"type": "image"},
            {"type": "text", "text": test_prompt}
        ]
    }
]

input_text = tokenizer.apply_chat_template(messages, add_generation_prompt = True)
inputs = tokenizer(
    test_image,
    input_text,
    add_special_tokens = False,
    return_tensors = "pt",
).to("cuda")

# 3. Sinh chuỗi giải toán từng bước
text_streamer = TextStreamer(tokenizer, skip_prompt = True)

print("=" * 60)
print(f"ĐỀ BÀI: {test_prompt}")
print("=" * 60)
print("MÔ HÌNH ĐÃ SFT BẮT ĐẦU GIẢI:")

_ = model.generate(
    **inputs,
    streamer = text_streamer,
    max_new_tokens = 1536, # Đủ dài để viết CoT và \boxed{}
    use_cache = True,
    temperature = 0.2, # Giảm về 0.2 để tính toán chặt chẽ, không bị ảo giác
    top_p = 0.95
)

```

$\frac{N}{M} \in \mathbb{Z}, \frac{M}{P} \in \mathbb{Z}, \frac{P}{Q} \in \mathbb{Z} \langle \text{im_end} \rangle$

Saving to float16 for VLLM

We also support saving to `float16` directly. Select `merged_16bit` for float16. Use `push_to_hub_merged` to upload to your Hugging Face account! You can go to <https://huggingface.co/settings/tokens> for your personal tokens. See [our docs](#) for more deployment options.

- float32 (FP32 - 32 bit = 4 byte): Độ chính xác tiêu chuẩn cũ, rất nét nhưng tốn bộ nhớ gấp đôi. Mô hình 4B tham số sẽ nặng khoảng 16 GB.
- float16 (FP16 - 16 bit = 2 byte): Chuẩn phổ biến nhất hiện nay cho các mô hình AI. Dung lượng giảm một nửa nhưng giữ nguyên gần như 100% độ chính xác. Mô hình 4B tham số sẽ nặng khoảng 8 GB.
- 4-bit (QLoRA bạn đang train): Nén tối đa xuống còn 4 bit (0.5 byte) mỗi trọng số để nhét vừa vào GPU T4 của Colab. Mô hình 4B chỉ tốn khoảng 4.5 GB VRAM.

Unsloth mới cộng toán học trực tiếp ma trận LoRA vào ma trận gốc ($W_{new} = W_{base} + \Delta W$) và xuất ra một thư mục nặng khoảng 8GB – 9GB chứa toàn bộ mô hình đã hợp nhất.

✓ KHÔNG NÊN CHẠY BƯỚC NÀY TRÊN GOOGLE COLAB MIỄN PHÍ.

- Dễ sập RAM: Quá trình giải nén 4-bit và hợp nhất ma trận sang 16-bit ngốn từ 14GB đến 18GB System RAM, rất dễ khiến máy ảo Colab (chỉ có 12.7GB RAM) bị tràn bộ nhớ và văng kernel.
- Tốn dung lượng: File sau khi lưu nặng tới hơn 8GB, rất lâu để tải về máy hoặc lưu vào Google Drive.

- Không cần thiết cho giai đoạn tiếp theo: Khi bạn chuyển sang làm Học tăng cường (RL / GRPO) ở Notebook 2, Unsloth hỗ trợ nạp thẳng thư mục adapter LoRA nhỏ gọn (qwen_geometry_sft_lora nặng ~100MB) mà không bắt buộc bạn phải gộp sang float16
- ```
Select ONLY 1 to save! (Both not needed!)

Save locally to 16bit
if False: model.save_pretrained_merged("unsloth_finetune", tokenizer,)

To export and save to your Hugging Face account
if False: model.push_to_hub_merged("YOUR_USERNAME/unsloth_finetune", tokenizer, token = "YOUR_HF_TOKEN")
```

## GGUF / llama.cpp Conversion

To save to GGUF / llama.cpp, we support it natively now! We clone llama.cpp and we default save it to q8\_0. We allow all methods like q4\_k\_m. Use save\_pretrained\_gguf for local saving and push\_to\_hub\_gguf for uploading to HF.

Some supported quant methods (full list on our docs page):

- q8\_0 - Fast conversion. High resource use, but generally acceptable.
- q4\_k\_m - Recommended. Uses Q6\_K for half of the attention.wv and feed\_forward.w2 tensors, else Q4\_K.
- q5\_k\_m - Recommended. Uses Q6\_K for half of the attention.wv and feed\_forward.w2 tensors, else Q5\_K.

[NEW] To finetune and auto export to Ollama, try our [Ollama notebook](#)

```
Save to 8bit Q8_0
if False: model.save_pretrained_gguf("qwen_finetune", tokenizer,)
Remember to go to https://huggingface.co/settings/tokens for a token!
And change hf to your username!
if False: model.push_to_hub_gguf("HF_USERNAME/qwen_finetune", tokenizer, token = "YOUR_HF_TOKEN")

Save to 16bit GGUF
if False: model.save_pretrained_gguf("qwen_finetune", tokenizer, quantization_method = "f16")
if False: model.push_to_hub_gguf("HF_USERNAME/qwen_finetune", tokenizer, quantization_method = "f16", token = "YOUR_HF_TOKEN")

Save to q4_k_m GGUF
if False: model.save_pretrained_gguf("qwen_finetune", tokenizer, quantization_method = "q4_k_m")
if False: model.push_to_hub_gguf("HF_USERNAME/qwen_finetune", tokenizer, quantization_method = "q4_k_m", token = "YOUR_HF_TOKEN")

Save to multiple GGUF options - much faster if you want multiple!
if False:
 model.push_to_hub_gguf(
 "HF_USERNAME/qwen_finetune", # Change hf to your username!
 tokenizer,
 quantization_method = ["q4_k_m", "q8_0", "q5_k_m",],
 token = "YOUR_HF_TOKEN",
)
```

And we're done! If you have any questions on Unsloth, we have a [Discord](#) channel! If you find any bugs or want to keep updated with the latest LLM stuff, or need help, join projects etc, feel free to join our Discord!

Some other resources:

1. Looking to use Unsloth locally? Read our [Installation Guide](#) for details on installing Unsloth on Windows, Docker, AMD, Intel GPUs.
2. Learn how to do Reinforcement Learning with our [RL Guide and notebooks](#).
3. Read our guides and notebooks for [Text-to-speech \(TTS\)](#) and [vision](#) model support.
4. Explore our [LLM Tutorials Directory](#) to find dedicated guides for each model.
5. Need help with Inference? Read our [Inference & Deployment page](#) for details on using vLLM, llama.cpp, Ollama etc.



unsloth



Join our Discord



Documentation

Join Discord if you need help + ★ Star us on [Github](#) ★

This notebook and all Unsloth notebooks are licensed [LGPL-3.0](#)